

Methodology

May 14, 2026

We propose a methodology for EPSS score generation for two scenarios: TPG-plus-GNN model, and TPG-plus-GNN-plus-tabular model extended with the processing of numerical and categorical vulnerability features. Both models employ TPG-generating backend based on spaCy[15] frontend. spaCy frontend performs tokenization of the raw text and creates a linguistic skeleton of the future TPG. Resulting tokenized representation preserves the original text structure, from which the original text is easily reconstructable by character offset. This alignment is crucial for the subsequent computation of embeddings of the nodes. Statistical component of spaCy perform type annotation of each token in four concrete types. Each token is annotated with 1) surface form, lemma, and a part-of-speech tag, 2) a link to the syntactic tag with a syntactic dependence relation (nsubj, dobj, amod, prep, etc.). 3) named entity recognition (PERSON, ORG, DATE, etc.). 4) sentence and noun-phrase chunking: each sentence becomes a SENTENCE node, noun chunks become NOUN-PHRASE. The tokenized, annotated text is then consumed by the first layer of the TPG backend, which organizes the annotated entities with their tags into nodes of the graph, and their dependence relations into edges of 13 generic types (DEP, NEXT TOKEN, NEXT SENT, NEXT PARA, COREF, SRL ARG, AMR EDGE, RST RELATION, DISCOURSE, CONTAINS, BELONGS TO, ENTITY REL, SIMILARITY). Second layer of the TPG generation applies a security pass to the existing graph, extending existing structure with 12 additional node types and 10 SEC * edge types. The extraction of security-specific node and edge types is rule based: CVE and CWE identifiers, CVSS score/severity and generic severity, present code snippets, and suggested remediations are extracted with regular expression strings, vendor, product names, attack vector, and vulnerability types are extracted with curated, keyword-based dictionary. For each constructed TPG, context embeddings are computed with SecBERT [16]. Embedding pass covers WordPiece [17] sub-tokens of spaCy tokens and attaches an embedding to each TOKEN node of the TPG. Since spaCy preserves the structure of the text during its own tokenization process, the final embeddings are perfectly aligned with the raw text. The resulting TPGs are fed into the multiview-GNN. Here, each view is one of 4 (5 if security pass is applied) edge-type partitions, for which view-specific Gated Convolutional Neural Network (GCNN) produces per-view embedding. After mean-and-max pooling, a graph has a single embedding, consumed by the

prediction head. Hybrid TPG-plus-GNN-plus-tabular model also consumes a vector of tabular vulnerability metadata.